Online Topic Modeling with Human-in-the-Loop Support: An Implementation-Centered Report on a BERTopic-Based Monitoring System

Krishna Gajera
College of Engineering
San Diego State University
kgajera1820@sdsu.edu

Baris Aksanli
College of Engineering
San Diego State University
baksanli@sdsu.edu

***Abstract*—Conventional topic modeling treats a corpus as static, yet many practical applications require continuous adaptation to evolving text streams. This paper presents a batch-recurring online topic modeling system that employs BERTopic as the primary operational model, with Latent Dirichlet Allocation (LDA) and Non-negative Matrix Factorization (NMF) serving as comparative baselines. The central engineering contribution is a *train-and-merge* update strategy: for each incoming time window a fresh BERTopic model is trained on the new batch, then merged with the currently deployed model via a similarity-thresholded integration, thereby preserving accumulated topic structure while admitting emergent themes. The platform further implements multi-signal drift detection—comprising topic prevalence change, embedding-space centroid shift, Jensen–Shannon keyword divergence, and topic birth/death tracking—to quantify model stability across windows. A human-in-the-loop layer supports topic relabeling, merging, version history, and rollback through an audited API. The complete system is orchestrated with Prefect, served through FastAPI, and visualized in a Streamlit dashboard. Evaluation on Twitter and AG News corpora using coherence (C_v), diversity, and silhouette metrics demonstrates that BERTopic consistently achieves higher coherence and diversity than both lexical baselines, while the multi-signal drift framework provides actionable temporal monitoring absent from standard topic modeling workflows.**

***Index Terms*—BERTopic, topic modeling, online analytics, LDA, NMF, drift detection, human-in-the-loop, dashboard systems**

I. Introduction

Topic modeling is often introduced through offline experiments in which a fixed corpus is analyzed once and the resulting topics are treated as static. That setup is useful for learning algorithms, but it does not match how many real analytical systems are expected to operate. In practical settings such as customer-support monitoring, social media analysis, or continuously updated content streams, the main challenge is not simply to fit a topic model once. Instead, the challenge is to repeatedly ingest new text, preserve useful topic structure from prior batches, track distributional changes over time, compare competing modeling approaches, and allow analysts to intervene when automatically generated topics are poorly named or semantically redundant.

This report presents the design and implementation of a batch-recurring online topic monitoring system centered on BERTopic. The project is motivated by the need to combine semantic topic discovery with operational reliability. Specifically, BERTopic is used as the deployed model for topic extraction and inference, while LDA and NMF are integrated as benchmark baselines to provide comparative evidence under a common evaluation framework.

The core technical idea is a *train-and-merge* temporal update strategy. For each incoming time window, a fresh BERTopic model is trained on the new batch and then merged with the currently deployed model. This strategy allows the system to preserve prior topic structure while incorporating emerging themes, and it aligns naturally with model versioning, rollback, and drift monitoring requirements. Beyond modeling, the implementation includes API-based serving, dashboard analytics, and human-in-the-loop (HITL) controls for topic relabeling and consolidation.

The main contributions of this project are as follows:

1) An end-to-end architecture for online topic intelligence, integrating ingestion, modeling, monitoring, and interaction layers.
2) A practical BERTopic update mechanism based on batch retraining and model merging for temporal adaptation.
3) A comparative three-model framework (BERTopic, LDA, NMF) with shared metrics for defensible model selection.
4) A multi-signal drift detection module using prevalence change, centroid shift, keyword divergence, and topic birth/death events.
5) A governed HITL workflow with audit logs, model version history, and rollback support.

The remainder of this paper is organized as follows. Section II reviews related work, including the baseline models. Section III presents the system overview and scope. Section IV describes the complete system architecture—orchestration, modeling, drift detection with formal definitions, human-in-the-loop governance, serving, and interaction—together with the unified pipeline algorithm. Section V presents experimental evaluation: corpora, preprocessing, evidence protocol, and

benchmark results. Sections VI–VIII discuss implementation strengths, limitations, and conclusions.

II. Related Work

A. Latent Dirichlet Allocation

LDA is a generative probabilistic model that represents each document as a mixture of latent topics and each topic as a distribution over words [2]. Training is typically performed via variational inference or collapsed Gibbs sampling. LDA remains a widely adopted baseline due to its strong theoretical foundations and ease of interpretation. However, it operates in bag-of-words space and relies on lexical co-occurrence, which limits its effectiveness on short, noisy, or paraphrastic text where surface-word overlap is sparse. In the present system, LDA is retained as a probabilistic baseline to provide a classical reference point for topic quality comparison. Its inclusion allows the evaluation to distinguish whether quality gains from the primary model are attributable to semantic embeddings or merely to preprocessing choices.

B. Non-negative Matrix Factorization

NMF decomposes a non-negative document-term matrix $\mathbf{V} \approx \mathbf{WH}$ into a document-topic matrix $\mathbf{W}$ and a topic-word matrix $\mathbf{H}$, subject to non-negativity constraints [3]. The resulting topic-word vectors tend to produce sharp, interpretable keyword groups. Unlike LDA, NMF is not probabilistic; it optimizes a reconstruction loss (typically Frobenius norm or KL divergence) with optional regularization. In practice, NMF can behave differently from LDA even on identical corpora, often yielding tighter lexical clusters but with greater sensitivity to vocabulary size and sparsity. The present implementation includes stability-oriented engineering decisions in the NMF path—such as fallback regularization and minimum-topic guards—to handle the practical difficulties of applying matrix factorization to small, sparse short-text batches.

C. BERTopic and Neural Topic Modeling

BERTopic addresses the semantic limitations of lexical methods by combining pre-trained sentence embeddings [4] with dimensionality reduction via UMAP [5], density-based clustering via HDBSCAN [6], and class-based TF–IDF topic representation [1] [10]. This pipeline groups documents by meaning rather than surface overlap, making it particularly suitable for evolving corpora with limited vocabulary consistency. The present work selects BERTopic as the operational model for this reason and uses LDA and NMF as complementary baselines to support a three-way comparative evaluation.

D. Online Operation, Drift, and Human Oversight

Most topic modeling workflows are offline and single-pass, whereas deployed systems require temporal updates, distributional monitoring, and analyst control. Practical MLOps patterns emphasize orchestration, reproducibility, and lifecycle observability [7], [8]. The present work contributes an implementation-focused integration: batch-recurring model updates via train-and-merge, multi-signal topic drift monitoring, and a human-in-the-loop interface for relabeling, topic consolidation, version tracking, and rollback. This positioning distinguishes the project from standalone modeling experiments by addressing both modeling quality and operational governance.

III. System Overview and Scope

This section formalizes the goal of the project and the precise scope of the implemented system. Let a corpus arrive as a sequence of time-indexed batches $\{\mathcal{B}_t\}_{t=1}^{T}$, where each batch $\mathcal{B}_t = \{d_i\}$ contains documents observed within a configured time window. The system is designed to (i) extract a human-interpretable topic structure from each window, (ii) maintain an evolving deployed topic model $\mathcal{M}_t$ across windows, and (iii) provide monitoring signals that quantify when topic structure or usage has shifted sufficiently to require analyst attention. The emphasis is operational: the output is not only a set of topics, but a repeatable lifecycle that supports ingestion, update, evaluation, monitoring, and governance.

A. Operational vs. comparative modeling

The implementation uses a role-separated modeling strategy. BERTopic is treated as the *operational* model: it is the model that produces the deployed topic catalog, supports inference on unseen text, and drives the dashboard-level analytics. In contrast, LDA and NMF are implemented as *comparative baselines*. They are executed to contextualize topic quality under a shared evaluation protocol (e.g., coherence, diversity, and clustering separability), but they are not used for serving or human-in-the-loop editing. This distinction is methodologically important because it prevents the evaluation from implicitly assuming that a single modeling family is correct by definition.

B. Temporal update principle: train-and-merge

The defining design choice for temporal operation is a batch-recurring *train-and-merge* update mechanism. For each new batch $\mathcal{B}_t$, the system trains a fresh BERTopic model $\mathcal{F}_t$ using the batch documents, then merges $\mathcal{F}_t$ with the currently deployed model $\mathcal{M}_{t-1}$ using a similarity-thresholded integration step. The resulting merged model becomes the new deployed state $\mathcal{M}_t$. Conceptually, this implements an engineering trade-off: the approach is simpler and more controllable than fully streaming optimization, while still enabling new topics to emerge and old topics to persist across windows. It also aligns with model governance, since each update produces a versionable artifact that can be archived and rolled back.

C. Monitoring and governance objective

Temporal operation makes monitoring a first-class requirement. The system therefore computes drift signals between consecutive deployed states to detect meaningful change. In the implemented design, drift is not reduced to a single scalar; instead, multiple complementary indicators are tracked,

including changes in topic prevalence (distribution shift), movement of topic centroids in embedding space (semantic shift), divergence of topic keyword distributions (descriptor shift), and topic birth/death events (structural shift). These signals support alerting and prioritization in the user interface. In addition, the system exposes human-in-the-loop controls that allow analysts to impose interpretability and consolidation constraints (topic relabeling and merging) while preserving an audit trail and version history.

D. Scope Boundaries

The system is intentionally scoped to a batch-recurring operational setting. First, updates are performed at window granularity rather than via continuous streaming optimization. Second, HITL support is strongest for relabeling and merging; topic splitting is not implemented as a fully automated semantic operation. Third, while the software stack is reproducible, complete experiment replay depends on access to the external raw datasets referenced in configuration. These boundaries define the valid interpretation of the experimental evidence presented later.

IV. System Architecture

The system is implemented as a layered, modular architecture composed of four principal layers: orchestration, modeling, drift detection, human-in-the-loop, serving and interaction. Each layer encapsulates a well-defined responsibility, communicates with adjacent layers through structured interfaces, and can be independently modified or extended. This separation of concerns is a deliberate engineering choice: it prevents tight coupling between data processing logic, model training routines, API contracts, and user-interface components.

Figure 1 presents the high-level layered architecture. The remainder of this section describes each layer in detail.

A. Orchestration Layer

The orchestration layer governs the temporal execution lifecycle of the system. It is implemented using Prefect [7], which expresses each pipeline run as a directed acyclic graph (DAG) of typed tasks with retries and structured error handling. Operationally, the pattern is *clock-driven then data-driven*: a schedule (cron or manual trigger) fires when a configured time policy elapses, which starts a new run and fixes a batch window $[t_{\text{start}}, t_{\text{end}})$ with a unique batch identifier. That trigger invokes the *data pipeline*—ingestion, cleaning, deduplication, and persistence into batch-scoped stores—so downstream stages always consume consistent, modeling-ready corpora rather than ad hoc files.

With artifacts in place, the *operational ML* path trains BERTopic on the current window (or seeds an initial model), then merges the batch model into the deployed catalog using a similarity threshold τ, yielding the updated model $\mathcal{M}_t$. Optional LDA and NMF baselines read the same cumulative corpus for comparable metrics. *Drift detection* then contrasts $\mathcal{M}_t$ with the previous state to populate alerts and dashboard

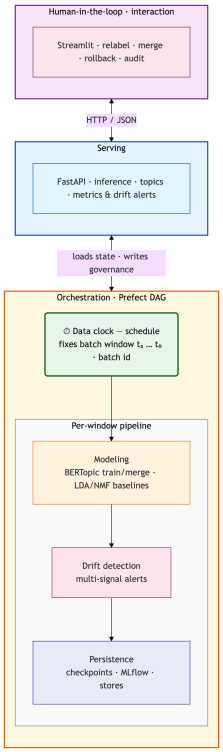

Fig. 1. Layered architecture of the online topic monitoring platform.

signals. Finally, *persistence* writes versioned checkpoints, assignments, and metadata so the FastAPI serving layer and Streamlit UI can load the new model without manual handoff. Experiment parameters, metrics, and artifact paths are logged to MLflow [8] throughout the run for traceability.

This design keeps the system batch-recurring: windows are processed in order, complexity stays bounded relative to streaming retraining, and scheduling remains configurable via cron expressions or on-demand execution.

B. Modeling Layer

The modeling layer encapsulates all topic model training, updating, and evaluation logic. It is organized around the role separation described in Section III: BERTopic operates as the primary model for discovery and deployment, while LDA and NMF serve as comparative baselines.

1) BERTopic modeling pipeline: Figure 3 follows the official BERTopic *modularity* illustration [1]: each pipeline stage is a swappable component—embeddings (e.g., SBERT, SpaCy, Hugging Face Transformers), dimensionality reduction (UMAP, PCA, truncated SVD), clustering (HDBSCAN, k-means, BIRCH), document-term extraction (CountVectorizer and alternatives), class-based TF–IDF variants (including BM25-style or normalized weighting), and optional representation layers (e.g., KeyBERT, MMR, or generative relabeling). Example vertical stacks in the figure show that valid pipelines are *compositions* of one block per row, not a single fixed recipe. The present system instantiates one such stack aligned with short-text semantic clustering: a pre-trained Sentence-

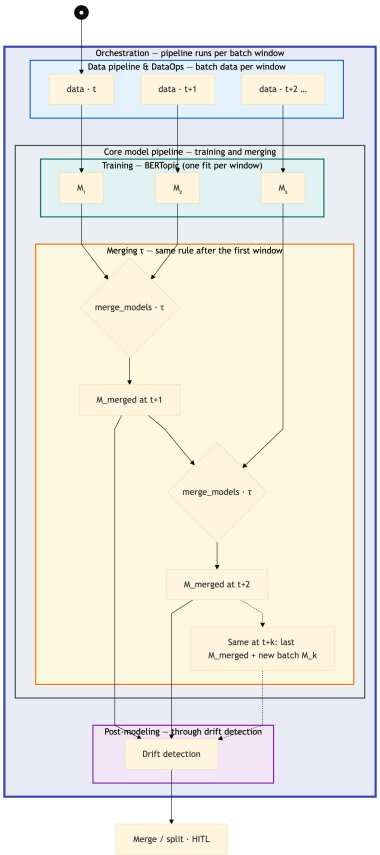

Fig. 2. Orchestration strategy: trigger and windowing, data pipeline, BERTopic train-and-merge, optional baselines, drift monitoring, and HITL handoff.

Transformer (`all-MiniLM-L6-v2`) [4], UMAP [5], HDBSCAN [6], CountVectorizer, and class-based TF–IDF [1]. After training, the layer extracts topic metadata (identifiers, top words, sizes, custom labels) and persists document-level topic assignments with confidence scores. Optional LLM-assisted label generation via a local Ollama endpoint can augment topic descriptors with natural-language summaries.

2) Online update via train-and-merge: When a deployed model $\mathcal{M}_{t-1}$ already exists, the modeling layer does not retrain on all historical data. Instead, it trains a fresh model $\mathcal{F}_t$ on the new batch $\mathcal{B}_t$, then invokes `BERTopic.merge_models` with a configurable similarity threshold τ to produce the updated model $\mathcal{M}_t$. Before merging, the previous model is archived to a versioned directory, enabling rollback. After merging, document assignments are recomputed using the merged model's `transform` method, and a cumulative corpus file is maintained for fair baseline comparison.

3) Topic metadata and interpretability: After each model update, the system extracts explicit topic metadata for every non-outlier topic: identifiers, ranked keywords, document count, time-window provenance, and human-readable labels. Optional LLM-assisted summarization can augment descriptors with natural-language topic names [9]. This metadata layer is essential because the dashboard, serving, and governance

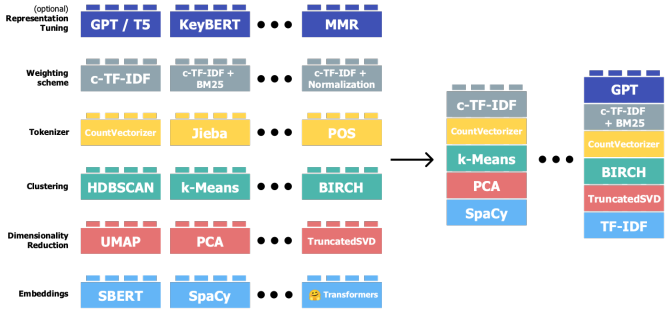

Fig. 3. BERTopic modularity: interchangeable blocks per stage (embeddings through c-TF–IDF and optional representation layers) and example composed pipelines; this work uses the Sentence-Transformer–UMAP–HDBSCAN–CountVectorizer–c-TF–IDF stack [1].

layers operate on curated topic descriptions rather than raw cluster assignments. The result is a reusable topic catalog that supports inspection, comparison, and editing across the full system lifecycle.

4) Baseline comparison path: LDA and NMF are trained on the same cumulative corpus available to the merged BERTopic model. The number of topics is harmonized: by default it matches the number of non-outlier topics discovered by BERTopic, with a configurable minimum. Both baselines record coherence (C_v), diversity, and silhouette metrics that are stored alongside BERTopic metrics for the same batch, enabling direct three-way comparison.

C. Drift Detection and Monitoring Layer

In a batch-recurring system, each model update potentially alters the topic structure. Without explicit monitoring, such changes accumulate silently and may degrade analytical reliability. The drift detection layer addresses this by computing quantitative signals between consecutive deployed model states $\mathcal{M}_{t-1}$ and $\mathcal{M}_t$ immediately after each merge. Rather than reducing drift to a single scalar, the implementation tracks four complementary indicators, each capturing a different manifestation of topic instability.

1) Prevalence drift: Let $P_t(k)$ denote the normalized prevalence of topic k in model $\mathcal{M}_t$, computed as the fraction of assigned documents. The system measures distributional shift using the Total Variation Distance (TVD):

$$\mathrm{TVD}(P_t, P_{t-1}) = \frac{1}{2} \sum_{k \in \mathcal{K}} |P_t(k) - P_{t-1}(k)| \tag{1}$$

where $\mathcal{K}$ is the union of all non-outlier topic identifiers across both models. Topics absent from one model are assigned zero prevalence. A high TVD indicates that the relative importance of topics has changed substantially between windows.

2) Centroid shift in embedding space: For each topic k present in both $\mathcal{M}_{t-1}$ and $\mathcal{M}_t$, the system computes the mean embedding vector (centroid) of all documents assigned to k in each window. The centroid shift is defined as:

$$\Delta_k = 1 - \frac{\mathbf{c}_t^{(k)} \cdot \mathbf{c}_{t-1}^{(k)}}{\|\mathbf{c}_t^{(k)}\| \, \|\mathbf{c}_{t-1}^{(k)}\|} \tag{2}$$

where $\mathbf{c}_t^{(k)}$ is the centroid of topic k in the embedding space of model $\mathcal{M}_t$. A value of $\Delta_k \approx 0$ indicates semantic stability; values approaching 1 indicate that the topic's meaning has shifted despite retaining the same identifier.

3) Keyword divergence: For each topic k, the system constructs a probability distribution over the union vocabulary of its top keywords in $\mathcal{M}_{t-1}$ and $\mathcal{M}_t$ by normalizing the c-TF-IDF weights. The divergence between these distributions is measured using the Jensen–Shannon divergence:

$$\mathrm{JSD}(Q_t^{(k)} \| Q_{t-1}^{(k)}) = \frac{1}{2}\,\mathrm{KL}(Q_t^{(k)} \| M) + \frac{1}{2}\,\mathrm{KL}(Q_{t-1}^{(k)} \| M) \quad (3)$$

where $M = \frac{1}{2}(Q_t^{(k)} + Q_{t-1}^{(k)})$ and $\mathrm{KL}(\cdot\|\cdot)$ denotes the Kullback–Leibler divergence. JSD is bounded in $[0, 1]$ and symmetric, making it suitable for comparing distributions that may have non-overlapping support.

4) Topic lifecycle events: Beyond continuous metrics, the system detects structural changes by computing set differences between the topic identifier sets of consecutive models:

$$\mathcal{K}_{\text{new}} = \mathcal{K}_t \setminus \mathcal{K}_{t-1}, \qquad \mathcal{K}_{\text{gone}} = \mathcal{K}_{t-1} \setminus \mathcal{K}_t \quad (4)$$

where $\mathcal{K}_t$ is the set of non-outlier topic identifiers in $\mathcal{M}_t$. A large $|\mathcal{K}_{\text{new}}|$ signals topic emergence; a large $|\mathcal{K}_{\text{gone}}|$ signals topic disappearance. Both are indicative of significant structural reorganization.

5) Alert generation and severity: Each drift signal is compared against configurable thresholds organized into three severity tiers (low, medium, high). When a signal exceeds its threshold, a timestamped alert is generated and persisted. Alerts carry topic-level granularity for centroid and keyword signals and system-level granularity for prevalence and lifecycle signals. This threshold-based design allows analysts to prioritize inspection effort based on signal severity rather than reviewing all topics after every update.

D. Human-in-the-Loop Governance Layer

Automated topic modeling produces topic structures that are statistically derived but not always analytically satisfactory. The governance layer addresses this by enabling analyst-driven interventions that are version-controlled and auditable, forming a feedback path from the interaction layer back into the deployed model state.

1) Relabeling: Topic relabeling allows an analyst to replace an automatically generated descriptor with a human-approved label. The operation modifies both the in-memory model and the persisted metadata, ensuring consistency between the deployed model and the dashboard. Each relabel action is recorded in an append-only audit log with a timestamp and optional analyst note.

2) Topic merging: When two or more topics are semantically redundant, an analyst can merge them into a single consolidated topic. The implementation invokes the model's merge operation on the full cumulative corpus, re-derives topic representations, and saves the updated model. The previous model state is archived before the merge, enabling rollback if the consolidation proves undesirable.

```
Algorithm 1: Batch-Recurring Online Topic Monitoring
Input: time window [t_s, t_e), config C
Output: updated model M_t, metrics, alerts
1. B_t ← Ingest and preprocess documents in [t_s, t_e)
2. if M_{t-1} exists then
3.    F_t ← Train fresh BERTopic on B_t
4.    M_t ← merge_models(M_{t-1}, F_t, τ)
5.    Archive M_{t-1} to versioned storage
6. else
7.    M_t ← Seed-train BERTopic on B_t
8. end if
9. Compute topic metadata and document assignments
10. Append B_t to cumulative corpus
11. if LDA enabled then Train LDA; record metrics
12. if NMF enabled then Train NMF; record metrics
13. if M_{t-1} exists then
14.    Compute prevalence TVD via Eq. (1)
15.    Compute centroid shift Δ_k via Eq. (2) ∀k
16.    Compute keyword JSD via Eq. (3) ∀k
17.    Compute K_new, K_gone via Eq. (4)
18.    Generate severity-graded drift alerts
19. end if
20. Log all metrics to MLflow; update processing state
21. Expose results via API for dashboard consumption
```

Fig. 4. Complete pipeline algorithm for one execution cycle. Drift steps (lines 14–17) reference the formal definitions in Section IV-D.

3) Version history and rollback: Every governance mutation (relabel, merge) and every pipeline-driven model update produces a versioned archive of the prior model state. The version history endpoint exposes a chronological list of archived checkpoints with metadata describing the action that produced each version. Rollback restores a selected archived checkpoint as the current deployed model. This mechanism provides a safety net that is essential in any system where human edits and automated updates coexist.

4) Implementation maturity: Topic splitting is exposed as an API endpoint but does not yet implement full semantic re-clustering. In its current form, the split interface records the analyst's intent and archives metadata, but it does not automatically discover coherent sub-topics within the source topic. This limitation is acknowledged as a validity boundary.

E. Complete Pipeline Algorithm

Figure 4 consolidates the full system behavior for a single execution cycle into a unified pseudocode representation. The algorithm integrates ingestion, the seed-or-merge modeling decision, baseline comparison, drift computation with explicit references to the formal metrics defined in Eqs. (1)–(4), alert generation, and state persistence.

F. Serving Layer

In a conventional research prototype, model outputs are consumed directly within the same process that produced them. This tight coupling limits reusability and makes it difficult to separate analytical concerns from training logic. The present system addresses this limitation by introducing a dedicated serving layer that mediates all access to model outputs, evaluation metrics, drift signals, and governance operations through a stateless RESTful API.

The API is implemented in FastAPI and organized into versioned resource groups that collectively cover six functional domains: topic catalog retrieval, temporal trend queries, drift alert access, single-document inference against the deployed BERTopic model, human-in-the-loop mutation operations (relabeling, merging, rollback), and per-model evaluation metric histories for BERTopic, LDA, and NMF. Each resource group returns structured JSON responses, enabling downstream consumers to be implemented independently of the modeling codebase.

A significant architectural benefit of this design is *consumer independence*. The dashboard, described in the following subsection, communicates with the serving layer exclusively through HTTP requests. However, the same API contracts are equally available to external scripts, automated monitoring agents, or alternative front-end applications. This decoupling also simplifies deployment: the backend and frontend are packaged as separate Docker containers orchestrated through Docker Compose, permitting independent scaling and lifecycle management.

G. Interaction Layer

The interaction layer is a multi-page Streamlit dashboard that provides analyst-facing views over the system's outputs. Each page corresponds to a distinct analytical concern:

1) **Project Overview** — system description, active dataset, and configuration summary.
2) **Dashboard** — current topic catalog with sizes, labels, and top-word lists.
3) **Topic Drilldown** — detailed inspection of individual topics, representative documents, and temporal prevalence.
4) **Drift Alerts** — active and historical drift signals with severity and metric detail.
5) **HITL Editor** — topic relabeling, merging, version history, and rollback controls.
6) **Inference** — free-text input for real-time topic prediction using the deployed model.
7) **Model Benchmarking** — side-by-side comparison of BERTopic, LDA, and NMF metrics across batches.

The dashboard communicates exclusively with the serving layer through HTTP requests to the FastAPI backend. This separation ensures that no modeling or storage logic is embedded in the front-end, which improves maintainability and permits the API to serve additional consumers (e.g., automated alerting scripts or external dashboards) without modification.

H. Persistence and Data Flow

Across all layers, persistence is managed through a shared file-system store organized by dataset name. Key artifacts include: processed Parquet files per batch, serialized BERTopic model checkpoints (current and previous), cumulative corpus JSON, topic metadata JSON, document assignment CSVs, drift alert CSVs, HITL audit logs, and per-model metric histories. MLflow additionally maintains a parallel experiment-tracking store for cross-run comparison.

Data flows unidirectionally from ingestion through modeling to serving and interaction, with the exception of HITL write-back operations (relabel, merge, rollback), which modify the deployed model and its metadata before the next pipeline cycle incorporates those changes through the merge process.

V. Experimental Evaluation

A. Datasets

Twitter customer-support conversations. We use a large-scale public corpus of Twitter-based customer support interactions (on the order of 2.8×10^6 raw records). The subset used for modeling focuses on short, user-originated messages after standard quality filters; the effective document count is substantially smaller than the raw table but remains in the millions-to-hundreds-of-thousands range depending on filtering. The text is inherently noisy: informal language, mentions, URLs, and heterogeneous brands in one stream. This domain is significant for topic modeling because lexical overlap is weak and paraphrase is common, which stresses semantic encoders more than clean newswire. Temporal batching uses strictly ordered timestamps where needed so window-based pipelines are well defined.

AG News. We additionally use the AG News collection: approximately 1.27×10^5 articles when standard train and test splits are combined, balanced across four high-level categories (world, sports, business, science and technology). Articles are longer and syntactically cleaner than social text, providing a complementary regime to test whether topic quality and model rankings transfer to traditional news-style language with stronger topical cues.

Quantitative benchmarks summarize five consecutive windows on each corpus, transcribed from the current project benchmark export.

B. Preprocessing

Preprocessing is profile-specific prior to ingestion into the pipeline. Support-style text uses aggressive noise removal (handles, URLs, boilerplate, punctuation clutter); news-style text uses lighter normalization to preserve headline and body phrasing. Empty and duplicate documents are dropped; minimum length thresholds reduce degenerate inputs. Document and batch identifiers are emitted for alignment with assignments, drift logs, and dashboard views. Raw corpora may not accompany the repository snapshot; evaluation assumes locally provisioned data as in deployment configuration.

C. Results

All models are evaluated under a common procedure: we report C_v coherence, top-term diversity, and silhouette computed in each model's native representation (silhouette is interpreted qualitatively across families). Topic count K is recorded per window for BERTopic and for baselines when aligned. LDA and NMF use the same cumulative corpus as BERTopic and, in automatic mode, the same K as non-outlier BERTopic topics as batches accrue. Batch identifiers and MLflow logs hold parameters, metrics, and timings for reproducibility.

TABLE I

PER-BATCH TOPIC METRICS FOR TWITTER SUPPORT (TOP) AND AG NEWS (BOTTOM). K IS THE BERTOPIC TOPIC COUNT; UNDER EACH MODEL, COLUMNS ARE C_v (COHERENCE), D (DIVERSITY), AND S (SILHOUETTE).

	Window	K	BERTopic			LDA			NMF		
			C_v	D	S	C_v	D	S	C_v	D	S
Twitter support	0010–15	35	.844	.951	.075	.653	.209	.042	.381	.305	.226
	0015–20	43	.760	.911	.020	.658	.342	−.001	.407	.363	.129
	0020–25	68	.837	.899	.040	.623	.268	−.003	.403	.410	.048
	0025–30	77	.853	.877	.033	.677	.286	−.016	.403	.461	.041
	0030–35	81	.769	.867	.006	.639	.280	−.008	.407	.524	.018
	Mean	—	.8126	.9010	.0348	.6503	.2768	.0029	.4003	.4125	.0924
AG News	0025–30	45	.863	.960	.212	.485	.500	.021	.568	.531	.146
	0030–35	71	.797	.913	.224	.451	.569	.054	.684	.687	.127
	0035–40	75	.868	.876	.179	.550	.607	.020	.742	.792	.143
	0040–45	84	.838	.850	.209	.503	.631	.012	.770	.825	.111
	0045–50	100	.818	.815	.140	.565	.566	.019	.780	.816	.098
	Mean	—	.8367	.8828	.1929	.5109	.5745	.0252	.7086	.7303	.1250

Table I gives batch-level scores; the last row of each block is the mean over its five windows. Aggregating those means, BERTopic achieves the strongest C_v and diversity on both corpora; NMF attains the highest mean silhouette on Twitter but lower C_v than BERTopic there, and on AG News ranks second on C_v without surpassing BERTopic. LDA shows the weakest mean C_v in both regimes but higher diversity on AG News than on Twitter. In the first Twitter window only, NMF used K=21 while BERTopic and LDA used K=35; later windows align K, and a stricter study would re-run mismatched cases under common K. Overall, the evidence supports BERTopic as the deployed model, with the caveat that silhouette must not substitute for coherence when rankings disagree. Negative LDA silhouette on some Twitter windows indicates weak separation in that representation for those batches.

VI. IMPLEMENTATION STRENGTHS

The project has several strengths that are especially relevant in a final review.

First, it treats topic modeling as a system rather than as an isolated algorithm. The implementation includes orchestration, monitoring, benchmarking, serving, and user interaction. This makes the project more substantial than a one-off modeling experiment.

Second, the choice of BERTopic as the operational model is well aligned with the type of data the system is intended to analyze. Short support-like texts benefit from semantic representations, and the implementation takes advantage of that while still preserving interpretable topic descriptors.

Third, the inclusion of LDA and NMF as real implemented baselines strengthens the evaluation. The project does not rely on a single-model argument. Instead, it uses comparison models to justify the modeling choice.

Fourth, the drift-monitoring and human-in-the-loop features show awareness of the practical lifecycle of topic models. In real usage, topics are rarely "done" after training. They require supervision, renaming, consolidation, and longitudinal monitoring. The project reflects that reality.

Finally, the dashboard and API design improve accessibility. The system is understandable not only to someone reading code, but also to someone interacting with it as an analytical product.

VII. LIMITATIONS AND AREAS FOR REFINEMENT

The project also has important limitations, and these are best presented directly rather than as vague future work.

The first limitation is reproducibility from the uploaded package alone. Although the codebase is substantial, the raw datasets and some generated outputs are not fully included, which means not every experiment can be rerun immediately from the archive.

The second limitation is partial inconsistency in the transition from a support-specific prototype toward a more general multi-dataset system. Some parts of the implementation clearly support multiple dataset profiles, but some surrounding assumptions still reflect an earlier customer-support-centered design. This does not invalidate the system, but it does indicate that the generalization process is not fully complete.

The third limitation concerns human-in-the-loop editing depth. Topic relabeling and merging are meaningfully implemented, but true topic splitting is not yet at the same level of maturity. This matters because split is conceptually harder than merge: it requires discovering coherent substructure within an existing topic rather than only consolidating redundant topics.

The fourth limitation concerns preprocessing consistency. The implementation contains dataset-aware cleaning, which is a strength, but end-to-end consistency must be maintained between training and inference for each dataset profile. Any mismatch there can weaken the reliability of live inference.

The fifth limitation is that model comparison metrics are informative but not perfectly unified across model families. Coherence, diversity, and silhouette each illuminate a different aspect of behavior, but they do not collapse into a single objective notion of quality. This means the evaluation should be presented as comparative evidence rather than as a definitive proof of superiority.

VIII. Conclusion

This project implements a strong final-year style topic modeling system centered on BERTopic and augmented with LDA and NMF benchmarking, drift monitoring, dashboard-based analytics, and human-in-the-loop maintenance. Its main technical value lies not in proposing a new topic model, but in operationalizing a modern semantic topic model inside a recurring analytical workflow.

The implementation makes several sound design choices. It uses BERTopic as the live model where semantic grouping and interpretability matter most. It benchmarks that model against two classical alternatives. It tracks batch-wise evolution over time rather than treating the corpus as static. It provides mechanisms for monitoring drift and for allowing analysts to correct topic structure. These are meaningful strengths for a project intended to demonstrate both machine learning understanding and systems integration.

At the same time, the system is best described precisely: it is a batch-wise online monitoring system, not a continuously streaming learner; it supports mature relabel and merge operations, but split remains partial; and its results are partly demonstrated through external run artifacts because the uploaded package is not fully self-contained. Presenting the project in this way is not a weakness. On the contrary, it shows technical maturity, because the implementation is explained in terms of what it truly does.

Acknowledgment

The author gratefully acknowledges Prof. Baris Aksanli (baksanli@sdsu.edu), graduate advisor, for sustained support on this project, including guidance on the overall definition and scope of the work, critical feedback during result validation, and practical help with system testing.

References

[1] M. Grootendorst, "BERTopic: Neural topic modeling with a class-based TF-IDF procedure," *arXiv preprint arXiv:2203.05794*, 2022.

[2] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet Allocation," *Journal of Machine Learning Research*, vol. 3, pp. 993–1022, 2003.

[3] D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, no. 6755, pp. 788–791, 1999.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[5] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," *arXiv preprint arXiv:1802.03426*, 2018.

[6] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Advances in Knowledge Discovery and Data Mining*, 2013, pp. 160–172.

[7] Prefect Technologies, "Prefect documentation," [Online]. Available: https://www.prefect.io/. Accessed: Mar. 2026.

[8] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[9] I. Al Azher, V. D. R. Seethi, A. P. Akella, and H. Alhoori, "LimTopic: LLM-based Topic Modeling and Text Summarization for Analyzing Scientific Articles limitations," in *Proc. 24th ACM/IEEE Joint Conf. Digital Libraries (JCDL)*, Dec. 2024, pp. 1–12, doi: 10.1145/3677389.3702605.

[10] Pavithra and Savitha, "Topic Modeling for Evolving Textual Data Using LDA, HDP, NMF, BERTOPIC, and DTM With a Focus on Research Papers," *Journal of Technology and Informatics (JoTI)*, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:269722938